Zero-Day Attack Framework Using Machine Learning-Based Anomaly Detection

Author One, Author Two, Author Three, Author Four
Department of Computer Science and Engineering
Mohan Babu University, India
{email1, email2, email3, email4}@example.edu

Abstract

Signature-based intrusion detection cannot recognise attacks it has never seen, and labelled data for novel attacks does not exist by definition. This work presents a practical framework for flagging zero-day network traffic by combining an unsupervised anomaly detector with a supervised classifier. An Isolation Forest, fitted only on benign flows, produces a calibrated anomaly percentile; this percentile is appended to the flow features and passed to a gradient-boosted tree classifier trained on SMOTE-balanced data. The two outputs are blended with a weight and a decision threshold that are tuned on a validation set containing an attack family withheld from training, and the blended score is mapped to a 0–100 risk score with severity bands. We evaluate with a leave-one-attack-family-out protocol on CIC-IDS2017 and compare six models on accuracy, precision, recall, F1-score and error rate. The proposed hybrid reaches an error rate of [run results_to_latex.py] and an F1-score of [run]; the best single baseline ([run]) reaches [run]. The detector is deployed behind a REST service and a web dashboard so analysts can upload flow files, review alerts and export reports.

Index Terms— zero-day attack, anomaly detection, intrusion detection, Isolation Forest, XGBoost, ensemble learning, CIC-IDS2017

1 Introduction

Network defenders increasingly face attacks for which no signature, rule or labelled training example exists. Classifiers trained on known attack classes tend to score unfamiliar traffic as benign, because the decision boundary was never shaped by that behaviour. Anomaly detectors take the opposite view: they model normal traffic and treat large deviations as suspicious, which allows novel attacks to be caught but at the price of false alarms on legitimate but unusual traffic [1].

Recent work has pursued representation learning on traffic graphs to score unseen malicious flows [2]. Such approaches are powerful but depend on graph construction and self-supervised pre-training that are heavy to reproduce and to operate. Our goal is more modest and operational: a framework built from well-understood components that a small security team could train on its own flow logs, evaluate honestly against unseen attack families, and run behind a web interface.

The contributions of this paper are:

- A hybrid scoring model that feeds a calibrated Isolation Forest percentile into a gradient-boosted classifier and blends both outputs, with the blend weight and threshold tuned on a pseudo-unknown attack family.
- A strict leave-one-attack-family-out evaluation in which every model, including the unsupervised ones, receives a threshold tuned on the same validation data.
- A risk-scoring layer that converts model scores into a 0–100 value, a severity level and a threat type (known attack, suspected zero-day, or suspicious anomaly).
- An end-to-end system: a Python inference service, an ASP.NET Core API with role-based access and auditing, and an MVC dashboard.

Problem statement. Given a stream of network flows in which some attack behaviours have never been labelled, flag the flows that deserve analyst attention, rank them by risk, and do so with a false-alarm rate low enough for an operations team to use. The objectives are therefore (i) to learn what normal traffic looks like, (ii) to separate known attacks from unfamiliar deviations, (iii) to evaluate honestly on attack families withheld from training, and (iv) to deliver results through a secure, auditable web application.

2 Related Work

Anomaly detection. Chandola et al. survey the field and distinguish point, contextual and collective anomalies [1]. One-class support vector machines learn a boundary around normal data [3]. Isolation Forest isolates observations with random axis-parallel splits and uses the short average path length of outliers as the anomaly signal [4]; it scales linearly with the number of samples, which suits flow-level data.

Supervised intrusion detection. Random forests [5], XGBoost [6] and LightGBM [7] are common baselines on tabular flow features and typically give high accuracy on attack classes present in training. Their weakness on unseen attack

types is what motivates evaluation protocols that withhold whole families.

Class imbalance. SMOTE synthesises minority-class examples by interpolating between neighbours [8]. Applied only to the training split, it reduces bias toward the majority class without touching evaluation data.

Datasets and their pitfalls. CIC-IDS2017 [9] is widely used, but Engelen et al. document labelling and flow-construction issues in it [10], so we clean the data and avoid claims that depend on individual labels. CTU-13 [11] provides botnet captures, and USTC-TFC2016 [12] provides malware and benign application traffic.

3 Datasets

CIC-IDS2017 is our primary dataset: five days of traffic with benign activity and attack families including DoS, DDoS, port scan, brute force, botnet, web attacks and infiltration, exported as flows with about 78 statistical features. **CTU-13** contains botnet scenarios captured as NetFlow records and is used for cross-dataset validation of the botnet family. **USTC-TFC2016** contains ten malware families and ten benign application classes captured as packet traces, which we convert to flows with CICFlowMeter. Each dataset is processed with its own feature space; models are not mixed across datasets. Table I summarises the sources.

Table I: Datasets used and their roles.

Dataset	Content	Role here
CIC-IDS2017	About 2.8 million labelled flows over five days, 78 flow features, benign plus 14 attack labels grouped into families	Primary: training, tuning, zero-day evaluation
CTU-13	13 botnet captures (several malware families) as NetFlow records with background, normal and botnet labels	Cross-dataset check of the botnet family
USTC-TFC2016	10 malware and 10 benign traffic classes as packet traces, converted to flows	Malware-versus-benign check

Data quality. Public flow exports contain infinite rates (for example bytes per second when a flow lasts zero microseconds), missing values, duplicated rows and trailing-space column names. These are handled in the cleaning stage and the number of rows removed at each step is logged so it can be reported with the results.

4 Proposed Framework

Fig. 1 shows the overall pipeline.

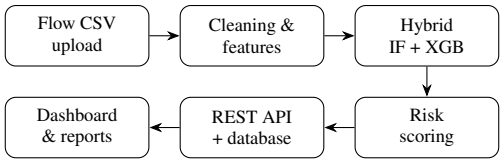


Figure 1: Processing pipeline from uploaded flows to analyst-facing alerts.

4.1 Cleaning and Feature Engineering

Column names are trimmed, non-numeric columns dropped, infinite values converted to missing values, duplicate rows removed and rows without a label discarded. A preprocessor fitted on the training split only imputes medians, winsorises each feature at the 0.1th and 99.9th percentiles, applies a signed logarithm $\mathrm{sgn}(x)\log(1+|x|)$ and standardises. Derived features capture forward/backward packet and byte ratios, bytes per packet, packets per millisecond and the SYN-minus-ACK flag difference. Features correlated above 0.95 are pruned, and the 40 features with the highest mutual information with the label are kept.

Table II lists the stages in the order they run, and Table III lists the engineered features.

Table II: Preprocessing pipeline (fitted on training data only).

Step	Stage	Operation
1	Clean	trim headers, drop non-numeric columns, infinity to missing, remove duplicates and unlabeled rows
2	Engineer	add ratio and rate features (Table III)
3	Impute	replace missing values with training medians
4	Winsorise	clip at 0.1th and 99.9th percentiles
5	Transform	signed logarithm, then standardise
6	Select	drop pairs correlated above 0.95; keep top 40 by mutual information

Table III: Engineered features.

Feature	Definition
pkt_ratio_fwd_bwd	forward packets / (backward packets + 1)
total_packets	forward + backward packets
byte_ratio_fwd_bwd	forward bytes / (backward bytes + 1)
total_bytes	forward + backward bytes
bytes_per_packet	total bytes / (total packets + 1)
pkts_per_ms	total packets / flow duration in ms
syn_minus_ack	SYN flag count − ACK flag count

4.2 Hybrid Scoring Model

Let x be a preprocessed flow. An Isolation Forest is fitted on benign training flows. Its raw anomaly score $r(x)$ is converted to an empirical percentile

$$s(x) = \frac{1}{|B|}\left|\{b \in B : r(b) \le r(x)\}\right|, \qquad (1)$$

where B is a set of out-of-fold benign scores, so that $s(x)$ is approximately uniform for normal traffic and close to 1 for strong outliers. Out-of-fold scores are used for benign training rows to avoid the optimistic bias of scoring data the forest was fitted on.

The classifier g is an XGBoost model trained on the augmented vector $[x, s(x)]$ after SMOTE balancing, yielding an attack probability $p(x)$. The final score is

$$f(x) = w\,p(x) + (1 - w)\,s(x), \qquad w \in [0, 1], \qquad (2)$$

and a flow is flagged when $f(x) \geq \tau$. The weight w is chosen from a grid and τ from the precision–recall curve to maximise F1 on a validation set that contains one attack family never seen during training. This lets the blend learn how much to trust the anomaly branch when the classifier has no knowledge of the attack.

4.3 Risk Scoring

The score is mapped piecewise linearly so that $f = \tau$ gives a risk of 50, $f = 0$ gives 0 and $f = 1$ gives 100. Severity bands are Low (< 50), Medium (50–69), High (70–89) and Critical (≥ 90). Flagged flows are labelled *Known Attack* when $p \geq 0.5$, *Zero-Day Attack* (suspected) when $p < 0.5$ but the anomaly percentile is at least 0.95, and *Suspicious Anomaly* otherwise. The reported confidence is the normalised distance from τ; it measures decision margin and is not a calibrated probability. Fig. 2 plots the mapping for three example thresholds: because the mapping is anchored at τ, a flow just above the learned decision boundary always lands at the start of the Medium band, whatever the absolute threshold.

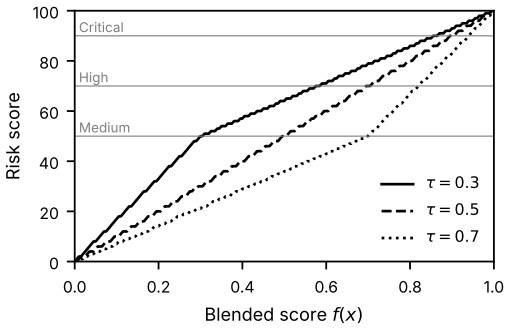


Figure 2: Risk score as a function of the blended score for example thresholds (generated from the system's risk-scoring function).

4.4 Model Configuration

Table IV gives the configuration of every model. All random seeds are fixed so that runs can be repeated.

4.5 Computational Cost

Isolation Forest training costs roughly $O(t\,\psi \log \psi)$ for t trees built on sub-samples of size ψ, and scoring one flow costs $O(t \log \psi)$, independent of the training-set size. Histogram-based gradient boosting costs roughly $O(K\,n\,d)$ per run for K trees, n rows and d features. Scoring is therefore linear in the number of uploaded flows, which is what allows a file of tens of thousands of flows to be analysed within a single web request. The out-of-fold calibration multiplies Isolation Forest training by the number of folds plus one, a one-off cost at training time.

Algorithm 1: Training the hybrid detector

Input: labelled flows D, held-out test family F
Output: model (g, w, τ) and risk mapping

1. Remove F from D; split benign and known attacks into train/validation; keep one known family V for validation only.
2. Fit the preprocessor and feature selector on the training split.
3. Fit an Isolation Forest on benign training flows; compute out-of-fold benign scores and the percentile table.
4. Append $s(x)$ to each training vector; balance classes with SMOTE; fit XGBoost to obtain $p(x)$.
5. For each w in a grid, compute f on the validation set and choose τ maximising F1; keep the best (w, τ).
6. Refit the classifier on all training data plus V, keeping (w, τ).
7. Evaluate on benign test flows plus all flows of F.

Figure 3: Training and evaluation procedure.

Table IV: Model hyper-parameters.

Model	Settings
Isolation Forest	200 trees; scores calibrated on 3-fold out-of-fold benign scores
One-Class SVM	RBF kernel, γ = scale, $\nu = 0.05$, fitted on up to 4,000 benign flows
Random Forest	200 trees
XGBoost	300 trees, depth 6, learning rate 0.1, subsample 0.8, column sample 0.8; optional randomised search (8 draws, 3-fold, F1) over trees, depth, rate, subsample, column sample and minimum child weight
LightGBM	300 trees, 63 leaves, learning rate 0.1, subsample 0.8, column sample 0.8
SMOTE	5 nearest neighbours, training split only
Hybrid	XGBoost as above on $[x, s(x)]$; w from {0, 0.1, …, 1}; τ from PR curve

Table V: Risk bands and alert logic.

Risk	Severity	Threat type (if flagged)
0–49	Low	Benign
50–69	Medium	Suspicious Anomaly, or Known Attack if $p \geq 0.5$
70–100	High, Critical (from 90)	Known Attack if $p \geq 0.5$; Zero-Day (suspected) if $p < 0.5$ and $s \geq 0.95$; otherwise Suspicious Anomaly

5 System Implementation

The trained pipeline is served by a FastAPI process that accepts a CSV file and returns per-flow results. An ASP.NET Core 8 Web API authenticates users with ASP.NET Identity and JWT bearer tokens, enforces three roles (Admin, Analyst, Viewer), validates uploads, calls the inference service, and stores datasets, predictions, threats, risk scores, alerts, reports and audit logs in SQL Server through Entity Framework Core. An ASP.NET Core MVC front end built with

Bootstrap and Chart.js presents totals, threat distribution, risk bands, detection trends and model metrics, and lets analysts acknowledge alerts and export PDF reports. Every upload, prediction, alert change and sign-in is written to an audit log.

Analyst workflow. A typical session has five steps: the analyst signs in and uploads a flow file; the server validates it and reports row counts, missing values and label distribution; the analyst starts an analysis, which sends the file to the inference service; the result page lists the highest-risk flows with their threat types and shows how risk is distributed across the file; and high-severity findings appear as open alerts that the analyst acknowledges or resolves, with each change recorded in the audit log. A threat or risk report for the analysis can then be exported as PDF for hand-over to other teams.

Database. Table VI lists the ten application tables. Deleting a dataset cascades to its flows, predictions, threats, risk scores and alerts; reports and audit entries survive deletion of their parent rows.

Table VI: Database tables.

Table	Purpose
Users, Roles	ASP.NET Identity accounts and roles
Datasets	uploaded file metadata, validation status and statistics
NetworkTraffic	sample of parsed flow fields per dataset
Predictions	one analysis run: totals, histogram, threshold, timing
RiskScores	per-flow risk, severity and confidence for flagged flows
Threats	flagged flows with type, severity and scores
Alerts	actionable alerts with Open, Acknowledged or Resolved status
Reports	stored threat and risk reports
AuditLogs	user, action, entity, time and address of each event

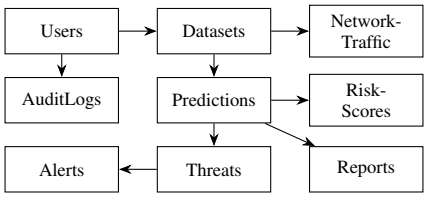


Figure 4: Simplified entity-relationship view; arrows point from parent to child. Users also own predictions, reports and alert acknowledgements.

Dashboard. Table VII describes the user-facing modules.

The prediction endpoint returns, for the highest-risk flow, a compact record such as the following (the values shown illustrate the format only and are not experimental output):

```
{ "riskScore": 96,
  "threatType": "Zero-Day Attack",
  "severity": "Critical",
  "confidence": "98%" }
```

Figure 5: Response format (illustrative values).

Interfaces. Table VIII lists the main REST endpoints, and Table IX shows what each role may do.

Table VII: Web application modules.

Module	Content
Dashboard	total flows, threats, high-risk alerts, zero-day detections; charts for threat distribution, risk bands, detection trend and model metrics
Datasets	upload, validation message, row and column counts, label distribution, first rows
Detection	analyse a new file or a stored dataset; headline result, risk histogram, top threats
Threats, Alerts	filtered lists; acknowledge or resolve alerts
Reports	generate threat or risk report; PDF export
Admin	role assignment, account activation, audit log

Table VIII: Main REST endpoints.

Endpoint	Function
POST /register, /login	create account, obtain JWT
POST /upload	upload and validate a flow CSV
GET /datasets	list datasets, with preview and statistics routes
POST /predict	analyse a file or stored dataset; returns risk score, threat type, severity, confidence
GET /threats, /alerts	filtered lists; alert status can be updated
GET /reports	list reports; create and export as PDF
GET /audit-logs	administrator audit trail

Table IX: Role permissions.

Action	Admin	Analyst	Viewer
View dashboard, threats, alerts, reports	yes	yes	yes
Upload data, run predictions	yes	yes	no
Update alerts, create reports	yes	yes	no
Delete datasets, manage users, view audit log	yes	no	no

Security. Passwords are hashed by ASP.NET Identity and must contain eight characters with upper-case, lower-case and a digit; five failed sign-ins lock an account for fifteen minutes. Tokens are signed with a secret supplied through the environment. The inference service is reached with a shared key over a private network, uploads are limited in size and type, and password-reset requests return the same message whether or not an account exists.

6 Experimental Setup

Environment. Table X lists the software stack.

Table X: Software environment.

Layer	Technology
Machine learning	Python 3, scikit-learn, XGBoost, LightGBM, imbalanced-learn, pandas, NumPy
Inference service	FastAPI served by Uvicorn
Backend	ASP.NET Core 8 Web API, Entity Framework Core, ASP.NET Identity, JWT bearer authentication
Database	SQL Server
Front end	ASP.NET Core MVC, Bootstrap 5, Chart.js
Packaging	Docker, Docker Compose; Render blueprint for hosting

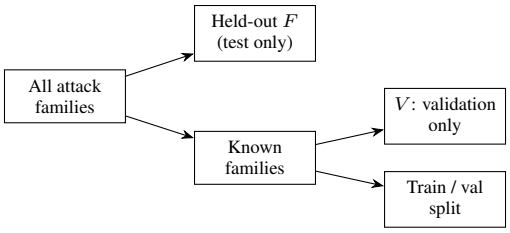


Figure 6: How attack families are allocated in each experiment.

Zero-day protocol. For each attack family F, all flows of F are removed from training and validation. Benign flows are split 60/20/20 into train, validation and test. Known attack families are split the same way, except that one of them is held out entirely as the pseudo-unknown validation family. The test set is the benign test portion plus every flow of F. Reported numbers are means over held-out families.

Split and validation strategy. Table XI shows how each family of flows is divided. Statistics for scaling and selection come from the training portion alone, so no information from validation or test flows reaches the preprocessing. Seeds are fixed, and the whole protocol can be repeated with different seeds to report variation.

Table XI: Data partitions per held-out family F.

Traffic	Train	Validation	Test
Benign	60%	20%	20%
Known attack families	60%	20%	—
Pseudo-unknown family V	—	100%	—
Held-out family F	—	—	100%

Threshold optimisation. For every model the decision threshold is the score at which F1 is highest on the validation precision–recall curve. The same rule is applied to the unsupervised and supervised models, so differences in the table come from the scores themselves rather than from a favourable default cut-off. Because the validation set contains an attack family that the classifier never saw, the chosen threshold reflects the situation the system faces at deployment.

Planned ablations. To show which design choice matters, the following variants will be evaluated under the same protocol (Table XII).

Models. Isolation Forest and One-Class SVM (trained on benign data only), Random Forest, XGBoost and LightGBM (trained on SMOTE-balanced data), and the proposed hybrid. Every model receives a decision threshold chosen by maximising F1 on the same validation set, so no model benefits from an untuned default.

Metrics. Accuracy, precision, recall, F1-score and error rate (one minus accuracy), together with false-positive rate, Matthews correlation coefficient, ROC-AUC and PR-AUC in the supplementary tables. XGBoost hyper-parameters are tuned by randomised search with three-fold cross-validation on F1.

Table XII: Ablation variants and the question each answers.

Variant	Question
No anomaly feature	Does appending $s(x)$ help the classifier?
$w = 1$ (classifier only)	How much does the blend add over the classifier alone?
$w = 0$ (anomaly only)	How far does unsupervised scoring go by itself?
No SMOTE	Does balancing change recall on unseen families?
No engineered features	Do ratio and rate features add information?
No feature selection	Is the top-40 filter helping or hurting?

With TP, FP, TN and FN denoting flagged attacks, false alarms, correct benign decisions and missed attacks,

$$\text{Precision} = \frac{TP}{TP + FP}, \qquad \text{Recall} = \frac{TP}{TP + FN}, \tag{3}$$

$$\text{F1} = \frac{2\,\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}, \qquad \text{Error} = \frac{FP + FN}{TP + FP + TN + FN}. \tag{4}$$

Because a hidden attack family is usually a minority of the test set, accuracy alone can hide missed attacks; recall and F1 are therefore reported next to error rate.

7 Deployment Architecture

The system runs as three containers and one managed database (Table XIII). The web application is the only component intended for end users; the API is also reachable for integration and exposes interactive documentation; the inference service is private and is called with a shared secret. Configuration, including connection strings, signing keys and the administrator account, is supplied through environment variables, never stored in the repository. Each public service exposes a health endpoint used by the host to restart failed instances, and uploaded files are kept on a persistent disk so that earlier datasets can be re-analysed after a redeployment.

Table XIII: Deployed components.

Component	Exposure	Responsibility
Web	public	MVC pages, session cookie holding the user's token
API	public	authentication, validation, persistence, reports, calls the inference service
Inference	private	loads the trained bundle; scores uploaded flows
Database	private	SQL Server tables of Table VI

The trained model is stored as a single serialised bundle containing the preprocessor, feature selector, classifier, anomaly scorer, blend weight and threshold, so that inference applies exactly the transformations fitted at training time. Retraining produces a new bundle, and the service is restarted to load it. Because the bundle is a pickled object, it must come

only from a trusted build process and never from user uploads.

8 Verification Plan

Table XIV lists the verification activities. Only the first has been carried out at the time of writing, on generated data used purely to confirm that the pipeline runs end to end; no performance claim is drawn from it.

Table XIV: Verification activities and status.

Activity	Scope	Status
Pipeline smoke test on generated flows	training, comparison, prediction, inference service	done
Zero-day experiments on real data	CIC-IDS2017, all families	pending
API endpoint tests	authentication, upload, predict, alerts, reports	pending
Role and access tests	permission matrix	pending
Large-file and malformed-CSV tests	upload validation	pending
Deployment health checks	web, API, inference service	pending

9 Results and Discussion

Table XV compares the six models under the zero-day protocol on CIC-IDS2017. The proposed hybrid has an error rate of [run results_to_latex.py], accuracy [run], precision [run], recall [run] and F1-score [run]. The strongest baseline by error rate is [run] at [run], giving a relative error change of [run] for the hybrid. Against the error rate reported for the base paper, [run], the relative change is [run].

Table XV: Zero-day comparison on CIC-IDS2017 (mean over held-out attack families; best value in bold).

Model	Acc.	Prec.	Rec.	F1	Err.
[run results_to_latex.py after model_compare.py]					

Table XVI breaks the error rate down by held-out family, which shows whether the hybrid helps uniformly or only on some attack types.

Table XVI: Error rate by held-out attack family on CIC-IDS2017 (best per column in bold).

[run results_to_latex.py after model_compare.py]

Threats to validity. Results depend on the label quality of CIC-IDS2017 [10]; treating each attack family as "unknown" only approximates truly novel attacks, because families share protocol-level behaviour; and the comparison with the base paper is indicative only, since its evaluation protocol and data splits differ from ours.

10 Limitations and Ethical Considerations

The framework inherits the weaknesses of its parts. An Isolation Forest trained on one network's benign traffic will raise alarms on legitimate behaviour that network has never produced, so thresholds must be re-tuned when a deployment changes. Flow statistics do not see payloads, so attacks that look like ordinary flows can be missed. The "zero-day" label produced by the system means unrecognised and anomalous, not confirmed novel; a human analyst should review every alert before action. Uploaded traffic can reveal user behaviour, so datasets should be anonymised, access is restricted by role, and the audit log records who viewed or changed what. Finally, public benchmark traffic is synthetic or laboratory-generated, and performance on it does not guarantee performance on production networks.

11 Conclusion and Future Work

We presented a hybrid anomaly-and-classification framework for detecting unseen attack traffic, a risk-scoring layer for analysts, and a deployable web system around it. Under a leave-one-attack-family-out protocol the hybrid achieved an error rate of [run results_to_latex.py] versus [run] for the best baseline. The accompanying source code, SQL scripts, container definitions and deployment guide allow the full system to be rebuilt from a clean checkout, and the experiment scripts regenerate every table in this paper directly from the saved result files. Future work includes cross-dataset evaluation on CTU-13 and USTC-TFC2016, online adaptation as traffic drifts, calibrated confidence estimates, explanations of which features drive a score, and evaluation on live captures.

References

[1] V. Chandola, A. Banerjee, and V. Kumar. Anomaly detection: A survey. *ACM Comput. Surv.*, 41(3):15:1–15:58, 2009.

[2] Yilmaz et al. A representation-and-scoring framework for zero-day malicious traffic detection using self-supervised graph learning. *IEEE Access*, 2026.

[3] B. Schölkopf, J. C. Platt, J. Shawe-Taylor, A. J. Smola, and R. C. Williamson. Estimating the support of a high-dimensional distribution. *Neural Comput.*, 13(7):1443–1471, 2001.

[4] F. T. Liu, K. M. Ting, and Z.-H. Zhou. Isolation forest. In *Proc. 8th IEEE Int. Conf. Data Mining (ICDM)*, pages 413–422, 2008.

[5] L. Breiman. Random forests. *Mach. Learn.*, 45(1):5–32, 2001.

[6] T. Chen and C. Guestrin. XGBoost: A scalable tree boosting system. In *Proc. 22nd ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining*, pages 785–794, 2016.

[7] G. Ke, Q. Meng, T. Finley, T. Wang, W. Chen, W. Ma, Q. Ye, and T.-Y. Liu. LightGBM: A highly efficient gradient boosting decision tree. In *Advances in Neural Information Processing Systems 30 (NIPS)*, pages 3146–3154, 2017.

[8] N. V. Chawla, K. W. Bowyer, L. O. Hall, and W. P. Kegelmeyer. SMOTE: Synthetic minority oversampling technique. *J. Artif. Intell. Res.*, 16:321–357, 2002.

[9] I. Sharafaldin, A. H. Lashkari, and A. A. Ghorbani. Toward generating a new intrusion detection dataset and intrusion traffic characterization. In *Proc. 4th Int. Conf. Information Systems Security and Privacy (ICISSP)*, pages 108–116, 2018.

[10] G. Engelen, V. Rimmer, and W. Joosen. Troubleshooting an intrusion detection dataset: the CICIDS2017 case study. In *Proc. IEEE Security and Privacy Workshops (SPW)*, pages 7–12, 2021.

[11] S. García, M. Grill, J. Stiborek, and A. Zunino. An empirical comparison of botnet detection methods. *Comput. Secur.*, 45:100–123, 2014.

[12] W. Wang, M. Zhu, X. Zeng, X. Ye, and Y. Sheng. Malware traffic classification using convolutional neural network for representation learning. In *Proc. Int. Conf. Information Networking (ICOIN)*, pages 712–717, 2017.